

LLM Based Behavior Tree Generation for Long-Horizon Robot Task Planning

Yunho Kim*, Seongjoon Park*, Soohyun Lee*, Jihoon Choi* and Inmo Jang
Korea Aerospace University, S.Korea

* These authors contributed equally to this work



Introduction

Motivation

- Behavior Trees (BTs) are increasingly used in robot task planning for their modularity and flexibility, yet generating them from natural language remains an open challenge.
- Existing LLM-based BT methods either rely on one-step generation or decompose task goals through **forward tree expansion**, lacking failure recovery mechanisms for unsatisfied preconditions in long-horizon tasks.
- Such iterative methods are resource-intensive, incurring significant **LLM query overhead and token consumption** that limits practical deployability.

Objectives

- Achieve structural failure recovery and reduced LLM queries within a unified framework via PPA (Precondition-Postcondition-Action) library caching
- Benchmark against representative baselines, demonstrating competitive success rates with significantly fewer queries and tokens.

Methodology

Framework Architecture

- The system operates in two stages: a **High-Level Planner** synthesizes the goal condition sequence, and a **Low-Level Planner** performs backward chaining to expand the tree upon condition failure.

High-Level Planner

- Translates the natural language instruction into goal conditions and organizes them into **sequential or parallel subtrees** to construct the initial BT.

Low-Level Planner(Backchaining)

- For a failed condition, the LLM identifies the action and preconditions that satisfy it and inserts them as a sub-tree.
- The procedure recurses on each new precondition, so the tree expands incrementally until all conditions are satisfied.

PPA Library Reuse

- The LLM is queried only to generate PPA entries; reusing the stored library therefore reduces query count and token usage.

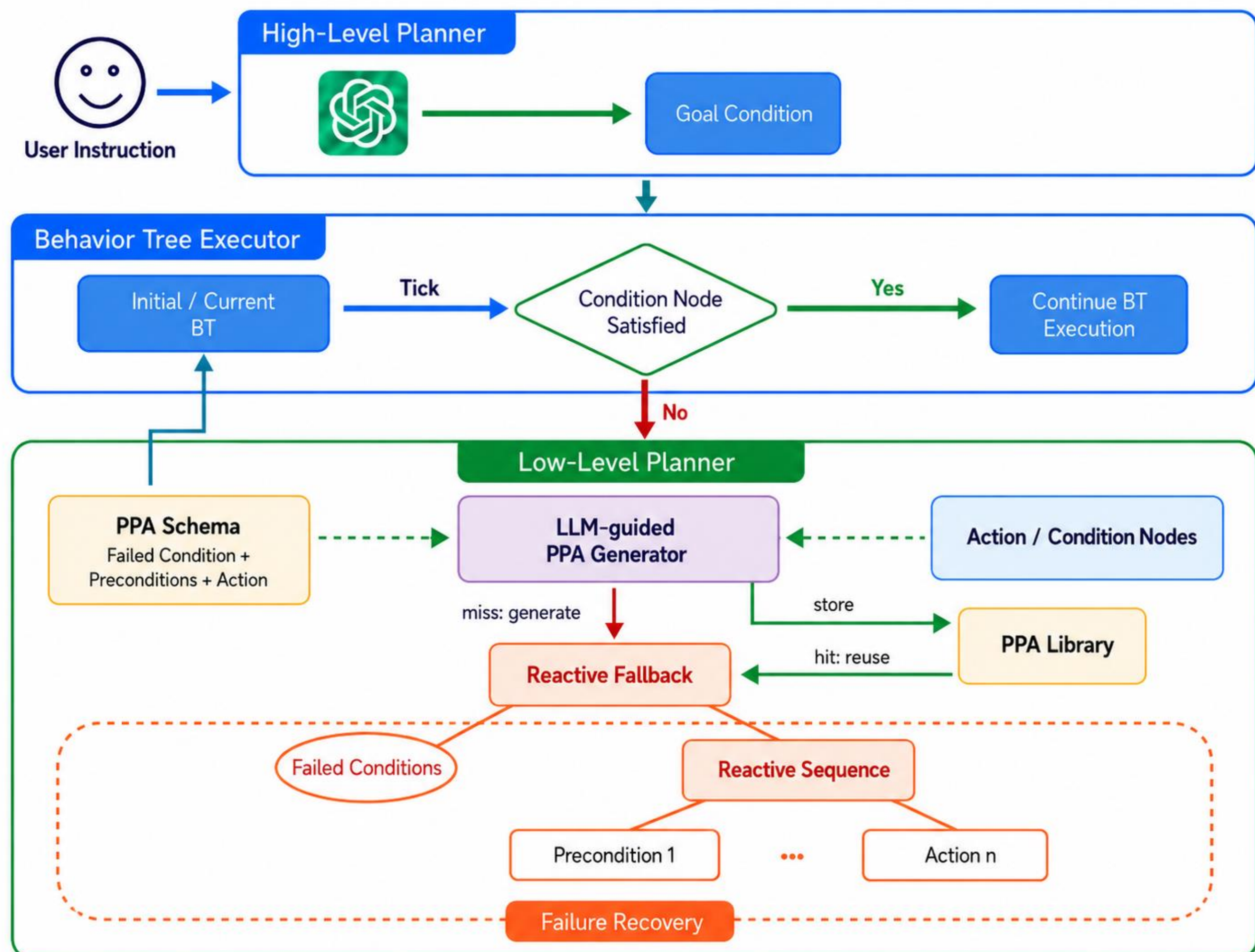


Fig. 1. Overall Architecture of the PPA-Based Behavior Tree Planning

Experiments

Setup

- Task:** Box-moving across 10 configurations (varying object/target positions) in a Pygame-based 2D simulator.
- Baselines:** One-step¹, STEP-Planner², Recursive³, and PPA-direct (PPA without caching).

1,3 : Ao et al., LLM as BT Planner, ICRA 2025 2 : Ao et al., STEP Planner, IROS 2025

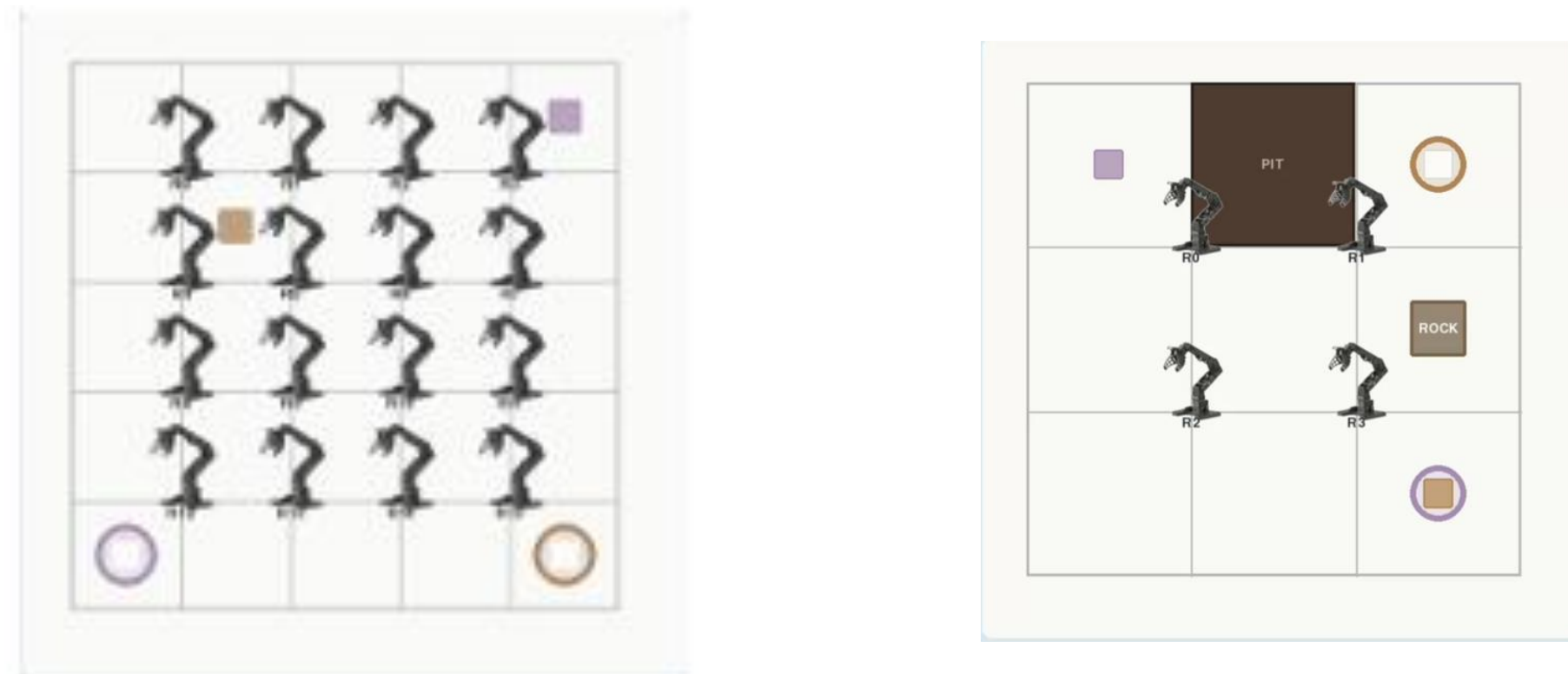


Fig. 2. Visualization in Pygame-based Simulator

Results

- One-step** (1 query): 73% Success Rate (SR) – single-pass generation cannot recover from conditions that fail during execution.
- STEP-Planner**: 0% SR at 334,297 tokens;
- Recursive**: auto-terminated due to runaway token usage.
- PPA-BT (Ours)**: **91% SR, -67% queries, -62% token consumption** relative to the best-performing baseline.

Method	Success Rate	Qurey	Tokens
One-step ¹	73%	1	12126
STEP-Planner ²	0%	109	334297
Recursive ³	-	-	-
PPA-direct	91%	45	126401
PPA-BT (Ours)	91%	15	47785

Table 1. Performance Comparison of Each Method in a 5 × 5 Environment (10 Trials)

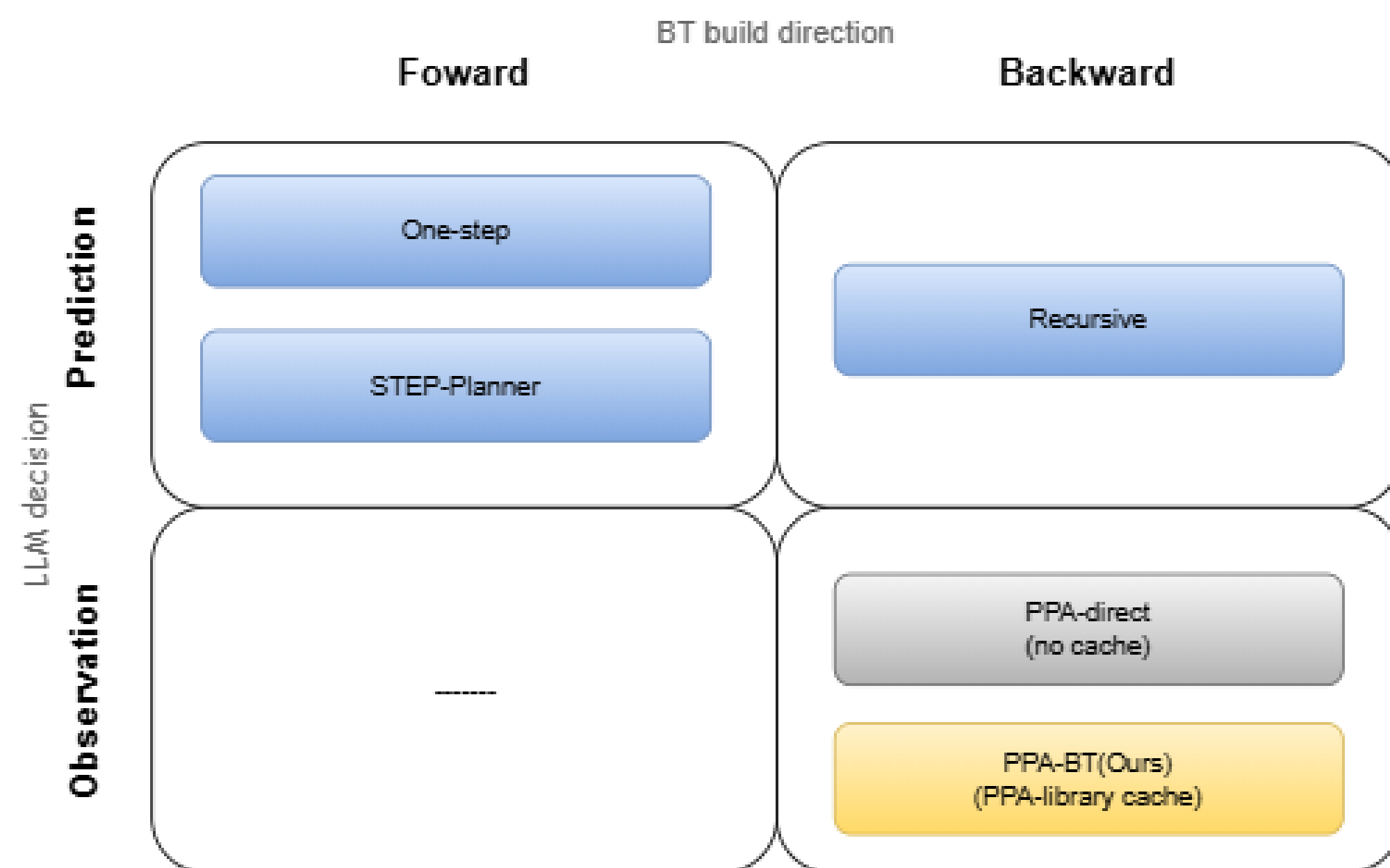


Fig. 3. Comparison Framework of LLM-Based Behavior Tree Planning Methods

Conclusion

- Backward chaining with condition-driven LLM queries** achieves higher and more consistent task success rates than forward-expansion or prediction-based approaches.

Future Work

- Extend validation to more complex environments (e.g., household chores) and multi-robot scenarios.
- Deploy the method on real robots such as humanoids and robotic arms for real-world testing.

